



BOOTCAMP FRONTEND & REACT · SEMANA 1

Flexbox

Clase 3 — Alinear y distribuir en una dimensión · Arranque de CSS Grid

L·M·V

Proyecto TechCart

STEERCL

LO QUE CUBRIMOS HOY (CON UN DESCANSO DE 15 MIN)

Agenda

1. Por qué Flexbox

El problema · contenedor, ítems y dos ejes · DevTools

2. El contenedor

justify-content, gap · align-items, el centrado clásico

3. Dirección y envoltura

flex-direction, flex-wrap, flex-flow, align-content · catálogo y aside

4. Los ítems

flex, align-self, order, margin auto · formulario, tarjeta y pie

5. Flexbox y Grid

Una dimensión frente a dos · arranque de Grid

6. Cierre

Errores frecuentes, resumen, tareas y anticipo

DE DÓNDE VENIMOS

Alinear con **bloque** y **en línea** no alcanza

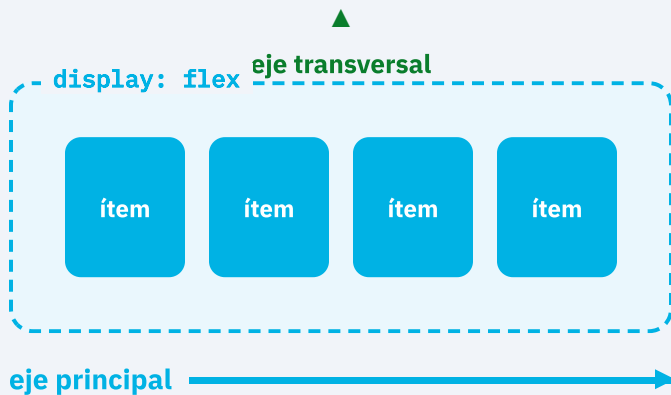


- 1 Poner dos cosas **lado a lado**
- 2 Separarlas a los **extremos**
- 3 Centrar algo en **vertical**
- 4 Que una fila **salte de línea** parejo

Las cuatro se resuelven hoy con Flexbox.

EL CONCEPTO

Contenedor, ítems y dos ejes



Contenedor

la caja de afuera: le ponemos `display: flex` (o `inline-flex` si va en línea)

Ítems

los hijos

directos

del contenedor, no los nietos

Eje principal

por defecto horizontal →

distribuir

a lo largo

Eje transversal

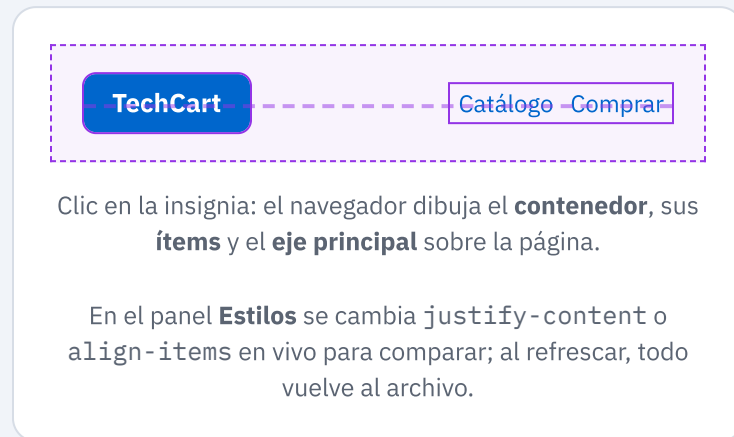
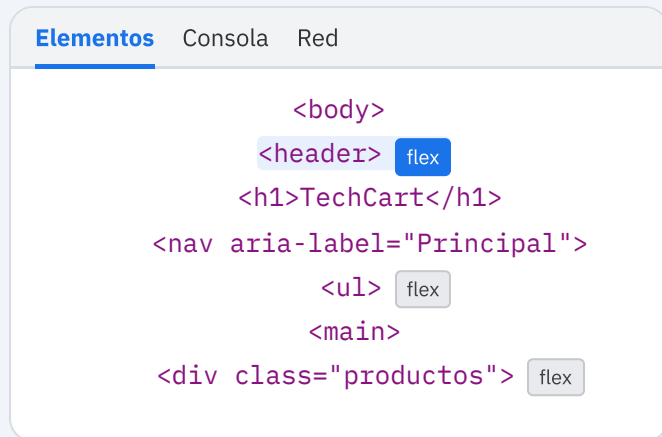
el perpendicular, vertical →

alinear

a lo alto

VER LOS EJES, NO ADIVINARLOS · F12

DevTools: la insignia **flex** y el overlay



DISTRIBUIR EN EL EJE PRINCIPAL · VA EN EL CONTENEDOR

justify-content

flex-start (defecto)



center



flex-end



space-between



space-around



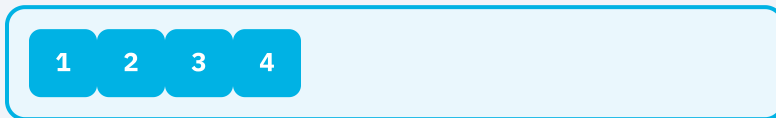
space-evenly



SEPARACIÓN UNIFORME ENTRE ÍTEMS · VA EN EL CONTENEDOR

gap · row-gap · column-gap

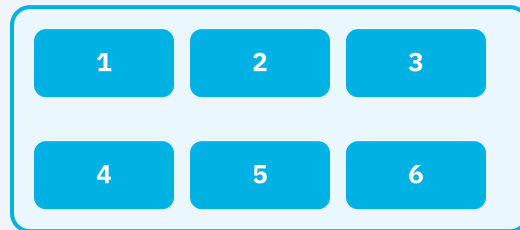
sin gap: pegados



gap: 14px: parejos, sin espacio en los bordes



gap: 22px 8px (filas · columnas)

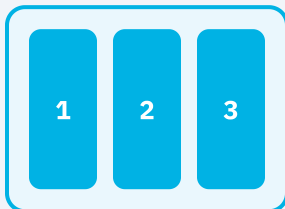


row-gap entre filas · column-gap entre columnas ·
gap es el atajo de los dos.

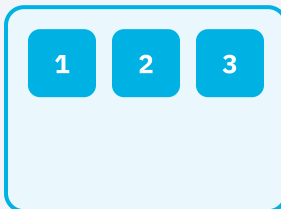
ALINEAR EN EL EJE TRANSVERSAL · VA EN EL CONTENEDOR

align-items

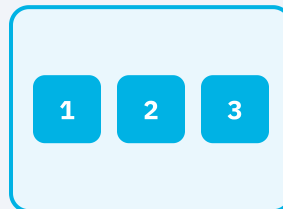
stretch (defecto)



flex-start



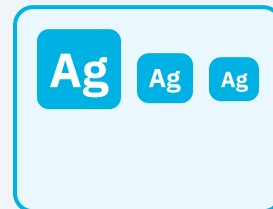
center



flex-end

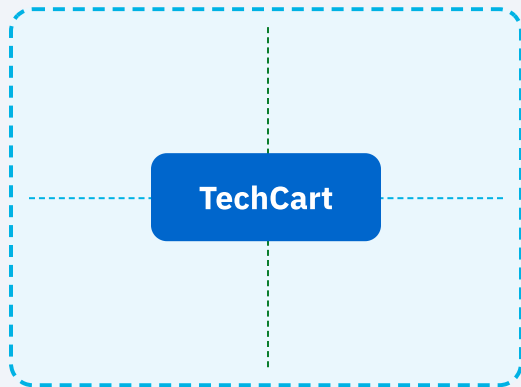


baseline



EL CASO CLÁSICO

Centrar en los dos ejes



```
header h1 {  
  display: flex;  
  justify-content: center; /* a lo largo */  
  align-items: center; /* a lo alto */  
  width: 140px; height: 44px;  
}
```

Lo que está adentro queda en el medio exacto, mida lo que mida la caja. El texto suelto dentro de un contenedor flex se trata como un ítem.

FILA O COLUMNA · AL CAMBIAR, LOS EJES ROTAN · ENVOLVER O APLASTAR

flex-direction · flex-wrap · flex-flow

row (defecto) →



row-reverse ←



column ↓



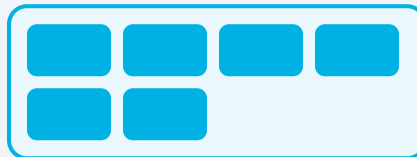
column-reverse ↑



flex-wrap: nowrap (defecto): se aplastan



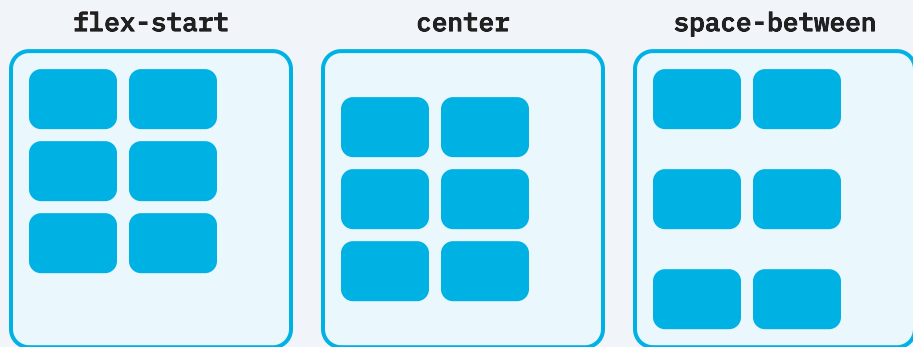
flex-wrap: wrap: saltan de línea



```
flex-flow: row wrap;
/* = flex-direction: row
+ flex-wrap: wrap */
```

CUANDO HAY VARIAS LÍNEAS (FLEX-WRAP) Y SOBRA ALTO

align-content: reparte las líneas



align-items

alinea los ítems
dentro
de su línea

align-content

reparte las
líneas

en el alto del contenedor (defecto:
stretch)

Solo se nota

si el contenedor es más alto que sus
líneas

VA EN CADA ÍTEM · GROW / SHRINK / BASIS

flex y el detalle de min-width: 0

flex-grow

¿crece para llenar lo que sobra? 0
no · 1 sí (2 = el doble)

flex-shrink

¿encoge si falta espacio? 1 sí
(defecto) · 0 nunca

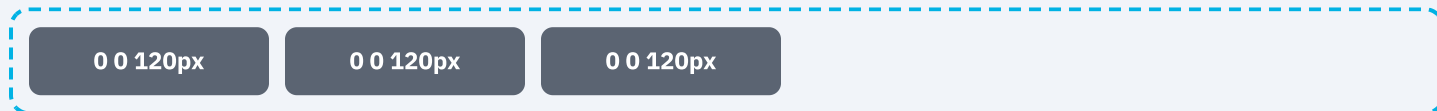
flex-basis

tamaño de partida (auto = su
tamaño natural o su width)

flex: 1 → crecen y se reparten en partes iguales



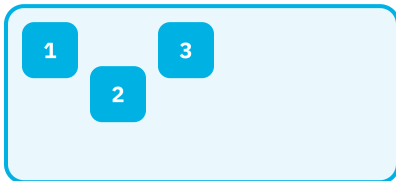
flex: 0 0 120px → no crecen, no encogen, miden 120 · **flex: 0 1 320px** → miden 320 y encogen si hace falta



Un ítem con texto largo sin
espacios no encoge por
debajo de su contenido y
desborda...
...salvo con min-width: 0



TRES PROPIEDADES DE ÍTEM MÁS

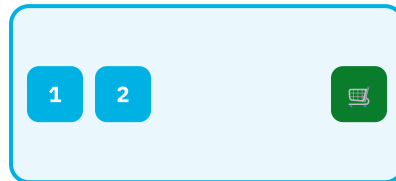
align-self • order • margin: auto**align-self**

Es `align-items` para **un solo** ítem: el contenedor dice cómo van todos, este dice cómo va él.
Mismos valores.

order

Cambia el **orden visual** sin tocar el HTML. Todos valen 0; menor va primero; acepta negativos.

Tab y lectores de pantalla siguen el HTML: solo para retoques.

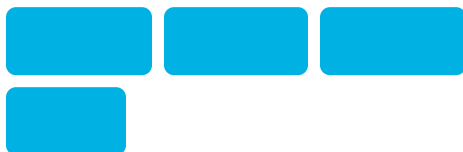
margin: auto

Se come **todo el espacio libre** y empuja al ítem al otro lado.
`margin-left: auto` → a la derecha (el carrito); `margin-top: auto` → al fondo (el botón de la tarjeta).

CADA UNO PARA LO SUYO, Y JUNTOS

Flexbox: **una** dimensión • Grid: **dos**

Flexbox • fila o columna



Cada línea se acomoda por su cuenta: la última fila queda despareja. Para **alinear contenido**: cabecera, menú, tarjeta, fila de formulario.

Grid • filas y columnas



Las columnas son las mismas en todas las filas: todo alineado. Para **maquetar**: cuadrícula del catálogo, esqueleto de la página, barra lateral.

DISPLAY: GRID · HAY QUE DECIRLE CUÁNTAS COLUMNAS

grid-template-columns · fr · repeat()

200px 200px 200px tres fijas



1fr 1fr 1fr tres que se reparten



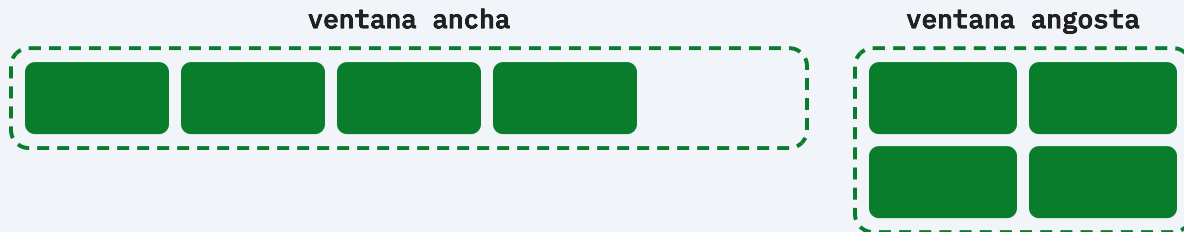
repeat(4, 1fr) cuatro iguales, sin repetir



fr = "una fracción del espacio que sobra". Es exclusiva de Grid y ya tiene el gap descontado. Sin `grid-template-columns`, Grid pone todo en una sola columna.

COLUMNAS QUE SE AGREGAN Y QUITAN SOLAS

```
repeat(auto-fill, minmax(200px, 1fr))
```

**minmax(200px, 1fr)**

cada columna
mide mínimo
200px y máximo
una fracción

auto-fill

mete tantas
columnas como
quepan

**Responsive sin
reglas por
tamaño**

una línea
reemplaza varias
reglas para móvil,
tablet y escritorio

LO QUE MÁS SE ROMPE AL EMPEZAR (Y CÓMO SE ARREGLA)

Errores **frecuentes** de Flexbox

1

Propiedad de contenedor en un ítem (o al revés): `justify-content` en un `<li>`, `flex: 1` en el `<ul>`.
Pregunta: ¿esto es para el padre o para un hijo?

2

Olvidar `flex-wrap`: `wrap` → las tarjetas se aplastan en una sola línea.

3

Confundir `justify-content` con `align-items` → `justify` a lo largo, `align` a lo alto; en columna rotan.

4

`align-items` que "no hace nada" → el contenedor no es más alto que su contenido; no hay espacio donde alinear.

5

`flex: 1` que desborda con texto largo sin espacios → falta `min-width: 0` en el ítem.

6

`display: flex` que "no hace nada" → está en el abuelo, no en el padre directo. Míralo en DevTools con el overlay.

LO QUE VIMOS HOY

Resumen

display: flex · inline-flex

enciende Flexbox en el contenedor;
ítems = hijos directos

flex-direction · flex-wrap · flex-flow

fila o columna (los ejes rotan) ·
envolver · el atajo

justify-content · gap

distribuir en el eje principal · separar
(row-gap, column-gap)

align-items · align-content

alinear en el transversal (stretch,
center, baseline...) · repartir líneas

flex · min-width: 0

grow / shrink / basis en cada ítem ·
dejarlo encoger de verdad

align-self · order · margin: auto

un ítem distinto · orden visual ·
empujar al extremo

display: grid

dos dimensiones: filas y columnas
alineadas

grid-template-columns · fr · repeat

cuántas columnas y de qué ancho

auto-fill · minmax

columnas que se adaptan solas; sigue
en la Clase 4

Tareas

F

Fácil: replica la clase completa, agrega un **quinto producto** y comprueba que la cuadrícula lo acomoda sola. De yapa, **flexboxfroggy.com** hasta el nivel 13.

I1

Intermedia: **barra de categorías** sobre el catálogo (`<nav>` + `<ul>` de 4 enlaces) en fila, centrada, con gap y flex-wrap.

I2

Intermedia: **formulario en dos columnas:** Ciudad + País y Teléfono + Fecha en un `<div class="fila">` flex, cada `<p>` con flex: 1.

D1

Difícil: el **botón de carrito:** "Comprar" como botón con un contador en un círculo de 22×22 centrado en los dos ejes; investiga inline-flex.

D2

Difícil: el **pie de una tienda** con tres columnas (marca, enlaces, contacto): flex: 1, align-items: flex-start, y que envuelvan con flex: 1 1 200px.

Próxima clase

Clase 4 — Terminar CSS Grid

Colocar cada cosa en una **celda**, que la tarjeta destacada ocupe **dos columnas** (span) y maquetar la página entera con **áreas con nombre**: la tienda gana su barra lateral de categorías.

Pista para casa: en `.productos`, cambia el `200px` del `minmax` por `150px` o `280px` y mira qué pasa con las columnas.